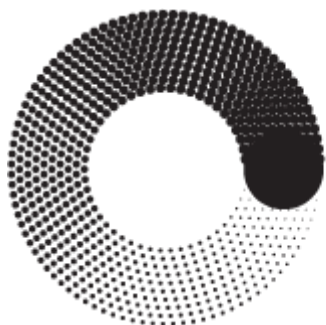


Министерство науки и высшего образования Российской Федерации  
Федеральное государственное автономное образовательное  
учреждение высшего образования



**МОСКОВСКИЙ  
ПОЛИТЕХ**

ОТЧЕТ

по Лабораторной работе 2

по дисциплине: «Технологии компьютерной графики»

|                 |                              |
|-----------------|------------------------------|
| Выполнил:       | Назаров Т. Д.                |
| Студент Группы: | 231-321                      |
| Проверили:      | Пухова Е.А.<br>Верещагин В.Ю |

Москва 2026

## 1. Цель работы

Изучить работу с цветовыми пространствами и каналами; реализовать инструменты для управления цветовыми каналами изображения и получения цвета отдельных пикселей в веб-приложении.

## 2. Постановка задачи

### 2.1 Панель каналов

Необходимо реализовать интерфейсный модуль для управления цветовыми каналами загруженного изображения. В панели должны отображаться миниатюры текущего изображения, разделенного на независимые каналы: Grayscale (оттенки серого), Grayscale + Alpha (оттенки серого с прозрачностью), RGB (три цветовых канала), RGB + Alpha (цветное изображение с прозрачностью). Всего должно быть четыре канала: Red (красный), Green (зеленый), Blue (синий) и Alpha (альфа-канал прозрачности).

Миниатюры должны наглядно демонстрировать содержимое каждого отдельного канала. Пользователь должен иметь возможность кликнуть на любой канал, чтобы включить или выключить его отображение. Состояние каждого канала (включен или выключен) должно быть очевидно для пользователя через визуальную индикацию (подсветку активного канала).

Изменение состояния в панели каналов должно мгновенно отражаться на главном холсте (canvas). Если пользователь отключает, например, зеленый канал, все пиксели на главном изображении должны потерять зеленую составляющую (значение зеленого компонента становится равным нулю). Если оставлен только альфа-канал, пользователь должен увидеть маску прозрачности в виде черно-белого изображения.

### 2.2 Инструмент «Пипетка»

Требуется создать инструмент для получения цвета конкретных пикселей изображения. Инструмент должен активироваться при выборе соответствующей кнопки в панели инструментов. При клике левой кнопкой мыши по любой точке изображения на главном холсте (canvas) приложение должно считывать данные выбранного пикселя.

На панели информации или в специальном всплывающем окне должны отобразиться следующие данные: координаты X и Y клика относительно самого изображения (а не относительно окна браузера), значения красного, зеленого и синего компонентов цвета в диапазоне от 0 до 255, а также координаты цвета в пространстве CIELAB (значения L, a, b).

При этом необходимо учитывать, что холст может быть отмасштабирован через CSS для того, чтобы поместиться в экран. Поэтому требуется вычислить масштабный коэффициент между реальным размером холста в пикселях и его отображаемым CSS-размером, чтобы точно сопоставить клик курсора с нужным индексом в массиве пикселей.

## 3. Теоретическая часть

### 3.1 Цветовые каналы

Цифровое изображение в формате RGBA представляется четырьмя каналами - красный (Red), зеленый (Green), синий (Blue) и альфа-канал (Alpha). Каждый канал хранит значение яркости соответствующей цветовой компоненты в диапазоне от 0 до 255, где 0 означает полное отсутствие компонента, а 255 - максимальную интенсивность.

При отключении определенного канала соответствующая цветовая компонента обнуляется, что позволяет визуально анализировать вклад каждого канала в итоговое изображение. Например, при отключении красного канала изображение приобретает голубовато-зеленый оттенок, а при отключении зеленого - розоватый.

### 3.2 Пространство CIELAB

CIELAB - это аппаратно-независимое цветовое пространство, разработанное Международной комиссией по освещению (CIE). Оно предназначено для того, чтобы воспринимаемая разница между цветами соответствовала евклидову расстоянию в этом пространстве.

Компонента L (Lightness) представляет светлоту цвета и принимает значения от 0 (черный) до 100 (белый). Компонента a (красно-зеленая ось) принимает положительные значения для красных оттенков и отрицательные для зеленых. Компонента b (сине-желтая ось) принимает положительные значения для желтых оттенков и отрицательные для синих. Диапазон значений a и b обычно составляет от -128 до +127.

### 3.3 Преобразование RGB в CIELAB

Преобразование цвета из пространства RGB в CIELAB происходит в два этапа:

Сначала RGB преобразуется в XYZ с учетом гамма-коррекции. Для этого каждый компонент RGB нормализуется в диапазон 0-1, после чего применяется гамма-коррекция по формуле - если значение меньше или равно 0.04045, оно делится на 12.92, иначе применяется степенная функция  $((\text{значение} + 0.055) / 1.055)^{2.4}$ . Затем полученные значения умножаются на 100 и преобразуются в XYZ с использованием матрицы преобразования.

Затем XYZ преобразуется в LAB с учетом опорной точки белого D65 ( $x_n = 95.047$ ,  $y_n = 100.000$ ,  $z_n = 108.883$ ). Для этого вычисляются отношения  $X/x_n$ ,  $Y/y_n$ ,  $Z/z_n$ , после чего применяется функция - если значение больше 0.008856, берется кубический корень, иначе используется линейная аппроксимация  $(7.787 * \text{значение} + 16/116)$ . Затем вычисляются значения L, a, b.

### 3.4 Визуализация каналов для предпросмотра

Для наглядного отображения миниатюр каналов используется подход с представлением каждого канала в градациях серого. При таком подходе белый цвет означает максимальную интенсивцию канала, а черный - отсутствие. Это делает интерфейс единообразным и понятным для пользователя.

Для каждого канала вычисляются минимальное и максимальное значения, после чего выполняется нормализация -  $\text{нормализованное значение} = (\text{исходное значение} - \text{минимум}) / (\text{максимум} -$

минимум)  $\times 255$ . Если все значения канала одинаковы (что бывает, например, с полностью черным или белым изображением), устанавливается среднее значение 128, чтобы миниатюра не была полностью черной или белой и сохраняла визуальную информативность.

## 4. Реализация

### 4.1 Архитектура модуля каналов

1. Состояние каналов

```
this.channels = { red: true, green: true, blue: true, alpha: true };
```

2. Применение каналов к изображению

```
405     applyChannels() {
406         if (!this.originalImageData) return;
407         const width = this.originalImageData.width;
408         const height = this.originalImageData.height;
409         const newImageData = new ImageData(width, height);
410         for (let i = 0; i < width * height; i++) {
411             const idx = i * 4;
412             let r = this.originalImageData.data[idx];
413             let g = this.originalImageData.data[idx + 1];
414             let b = this.originalImageData.data[idx + 2];
415             const a = this.originalImageData.data[idx + 3];
416             if (!this.channels.red) r = 0;
417             if (!this.channels.green) g = 0;
418             if (!this.channels.blue) b = 0;
419             newImageData.data[idx] = r;
420             newImageData.data[idx + 1] = g;
421             newImageData.data[idx + 2] = b;
422             newImageData.data[idx + 3] = this.channels.alpha ? a : 255;
423         }
424         this.currentImageData = newImageData;
425         this.renderCanvas();
426         this.updateChannelPreviews();
427     }
```

### 4.2 Генерация миниатюр каналов

```

474 generateChannelPreviews() {
475   if (!this.originalImageData) return;
476   const w = this.originalImageData.width, h = this.originalImageData.height;
477   let minR=255,maxR=0, minG=255,maxG=0, minB=255,maxB=0, minA=255,maxA=0;
478   for (let i = 0; i < w * h; i++) {
479     const idx = i * 4;
480     const r = this.originalImageData.data[idx];
481     const g = this.originalImageData.data[idx+1];
482     const b = this.originalImageData.data[idx+2];
483     const a = this.originalImageData.data[idx+3];
484     minR=Math.min(minR,r); maxR=Math.max(maxR,r);
485     minG=Math.min(minG,g); maxG=Math.max(maxG,g);
486     minB=Math.min(minB,b); maxB=Math.max(maxB,b);
487     minA=Math.min(minA,a); maxA=Math.max(maxA,a);
488   }
489   const channels = [
490     { name: 'RED', key: 'red', get: (r)=>r, min: minR, max: maxR },
491     { name: 'GREEN', key: 'green', get: (r,g)=>g, min: minG, max: maxG },
492     { name: 'BLUE', key: 'blue', get: (r,g,b)=>b, min: minB, max: maxB },
493     { name: 'ALPHA', key: 'alpha', get: (r,g,b,a)=>a, min: minA, max: maxA }
494   ];
495   const grid = document.getElementById('channelsGrid');
496   if (!grid) return;
497   grid.innerHTML = '';
498   channels.forEach(ch => {
499     const previewCanvas = document.createElement('canvas');
500     previewCanvas.width = 70;
501     previewCanvas.height = 70;
502     const pctx = previewCanvas.getContext('2d');
503     const pdata = pctx.createImageData(70, 70);
504     for (let y = 0; y < 70; y++) {
505       for (let x = 0; x < 70; x++) {
506         const sx = Math.floor(x / 70 * w);
507         const sy = Math.floor(y / 70 * h);
508         const idx = (sy * w + sx) * 4;
509         let val = ch.get(this.originalImageData.data[idx], this.originalImageData.data[idx+1], this.originalImageData.data[idx+2], this.originalImageData.data[idx+3]);
510         if (ch.max > ch.min) val = (val - ch.min) / (ch.max - ch.min) * 255;
511         else val = 128;
512         const didx = (y * 70 + x) * 4;
513         pdata.data[didx] = val;
514         pdata.data[didx+1] = val;
515         pdata.data[didx+2] = val;
516         pdata.data[didx+3] = 255;
517       }
518     }
519     pctx.putImageData(pdata, 0, 0);
520     const card = document.createElement('div');
521     card.className = `channel-card ${this.channels[ch.key] ? 'active' : ''}`;
522     card.innerHTML = `<div class="channel-name">${ch.name}</div>`;
523     card.onclick = () => {
524       this.channels[ch.key] = !this.channels[ch.key];
525       this.applyChannels();
526       card.classList.toggle('active', this.channels[ch.key]);
527     };
528     grid.appendChild(card);
529   });
530 }

```

- Находим мин/макс для нормализации – 477 строка
- Создаем миниатюру 70×70 – 504 строка.
- Запись значения во все RGB компоненты (градации серого)

### 4.3 Инструмент «Пипетка» с учетом масштаба

```

282 handleCanvasClick(e) {
283     if (!this.eyedropperActive || !this.currentImageData) return;
284     const rect = this.canvas.getBoundingClientRect();
285     const scaleX = this.canvas.width / rect.width;
286     const scaleY = this.canvas.height / rect.height;
287     const mouseX = (e.clientX - rect.left) * scaleX;
288     const mouseY = (e.clientY - rect.top) * scaleY;
289     const x = Math.floor(Math.max(0, Math.min(mouseX, this.currentImageData.width - 1)));
290     const y = Math.floor(Math.max(0, Math.min(mouseY, this.currentImageData.height - 1)));
291     const idx = (y * this.currentImageData.width + x) * 4;
292     const r = this.currentImageData.data[idx];
293     const g = this.currentImageData.data[idx + 1];
294     const b = this.currentImageData.data[idx + 2];
295     const lab = ColorConverter.rgbToLab(r, g, b);
296     document.getElementById('coordX').textContent = x;
297     document.getElementById('coordY').textContent = y;
298     document.getElementById('colorR').textContent = r;
299     document.getElementById('colorG').textContent = g;
300     document.getElementById('colorB').textContent = b;
301     document.getElementById('colorL').textContent = lab.L;
302     document.getElementById('colorA').textContent = lab.a;
303     document.getElementById('colorB2').textContent = lab.b;
304     document.getElementById('colorPreview').style.backgroundColor = `rgb(${r}, ${g}, ${b})`;
305     this.colorInfo.style.display = 'flex';
306     this.showToast(`↑ (${x},${y}) RGB(${r},${g},${b}) Lab(${lab.L},${lab.a},${lab.b})`, 'success');
307 }
308

```

#### 4.4 Конвертер RGB в CIELAB

```

65 class ColorConverter {
66     static rgbToXyz(r, g, b) {
67         let rr = r / 255, gg = g / 255, bb = b / 255;
68         rr = rr > 0.04045 ? Math.pow((rr + 0.055) / 1.055, 2.4) : rr / 12.92;
69         gg = gg > 0.04045 ? Math.pow((gg + 0.055) / 1.055, 2.4) : gg / 12.92;
70         bb = bb > 0.04045 ? Math.pow((bb + 0.055) / 1.055, 2.4) : bb / 12.92;
71         rr *= 100; gg *= 100; bb *= 100;
72         const x = rr * 0.4124564 + gg * 0.3575761 + bb * 0.1804375;
73         const y = rr * 0.2126729 + gg * 0.7151522 + bb * 0.0721750;
74         const z = rr * 0.0193339 + gg * 0.1191920 + bb * 0.9503041;
75         return { x, y, z };
76     }
77
78     static xyzToLab(x, y, z) {
79         const xn = 95.047, yn = 100.000, zn = 108.883;
80         let fx = x / xn, fy = y / yn, fz = z / zn;
81         fx = fx > 0.008856 ? Math.pow(fx, 1/3) : (7.787 * fx) + (16 / 116);
82         fy = fy > 0.008856 ? Math.pow(fy, 1/3) : (7.787 * fy) + (16 / 116);
83         fz = fz > 0.008856 ? Math.pow(fz, 1/3) : (7.787 * fz) + (16 / 116);
84         const L = (116 * fy) - 16;
85         const a = 500 * (fx - fy);
86         const b = 200 * (fy - fz);
87         return { L: Math.round(L), a: Math.round(a), b: Math.round(b) };
88     }
89
90     static rgbToLab(r, g, b) {
91         const { x, y, z } = this.rgbToXyz(r, g, b);
92         return this.xyzToLab(x, y, z);
93     }
94 }

```

## 5. Интерфейс пользователя

### 5.1 Панель каналов

Панель каналов расположена под основной областью редактирования и содержит четыре карточки для каждого канала (RED, GREEN, BLUE, ALPHA). Каждая карточка включает миниатюру канала размером  $70 \times 70$  пикселей и название канала. Активный канал подсвечивается золотой рамкой и фоном, неактивный - имеет темную рамку. При клике на карточку состояние канала переключается, и изображение на главном холсте мгновенно обновляется.

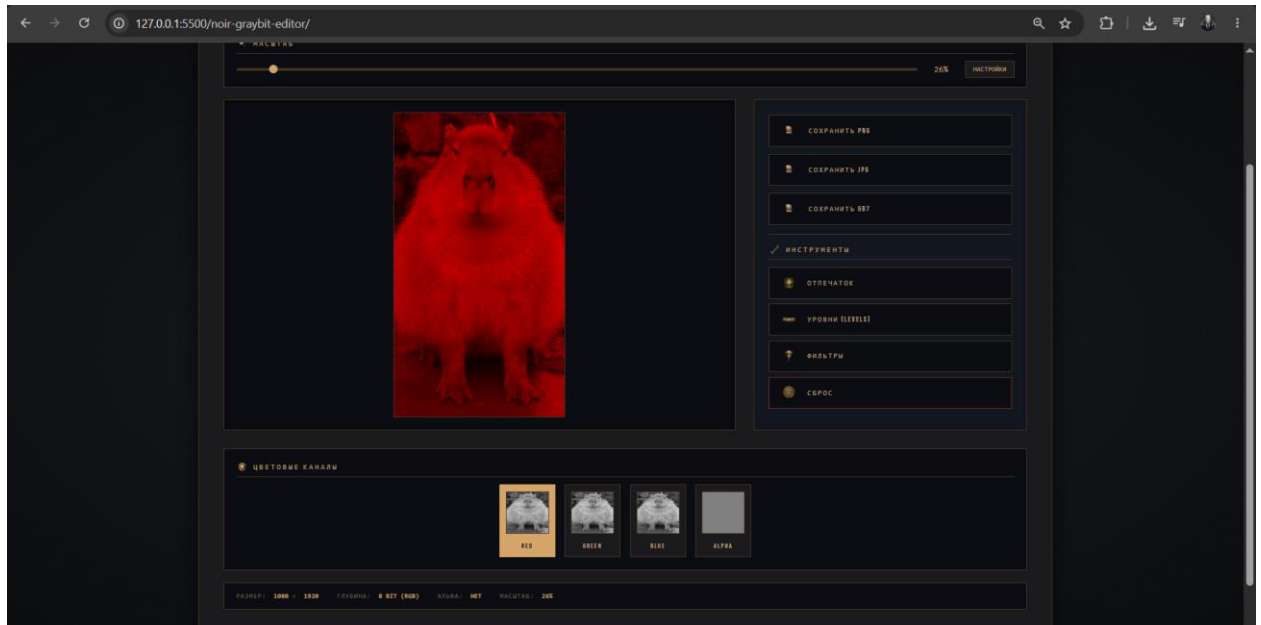


Рисунок 1 – канал RED

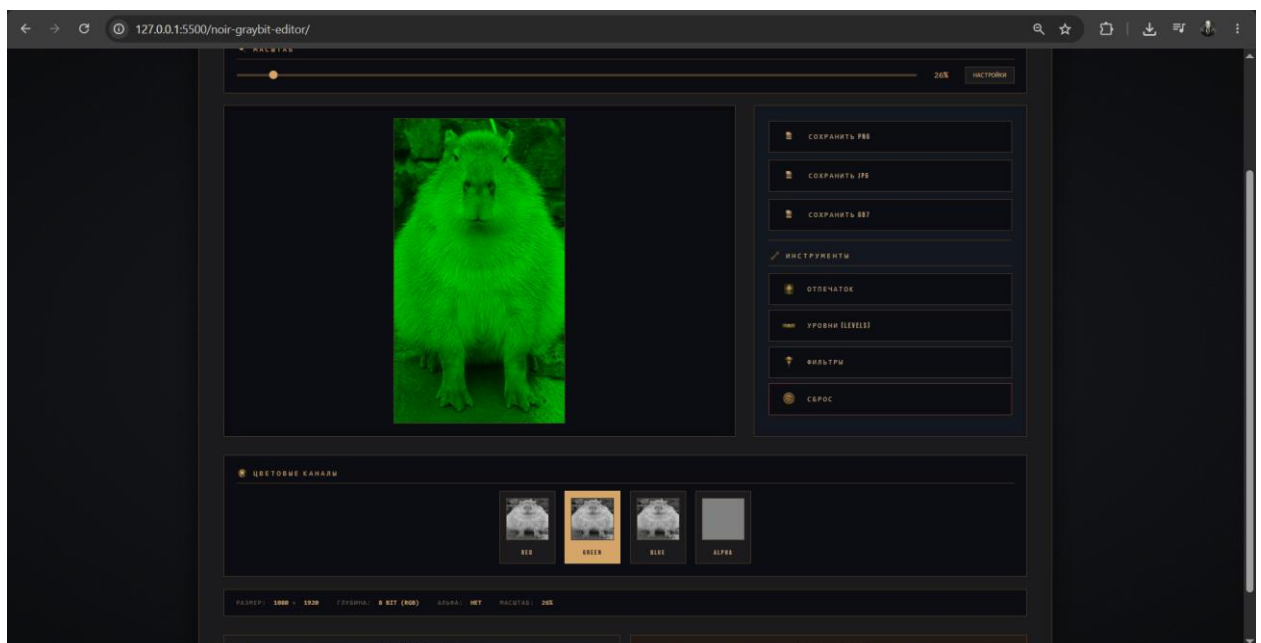


Рисунок 2 – канал GREEN

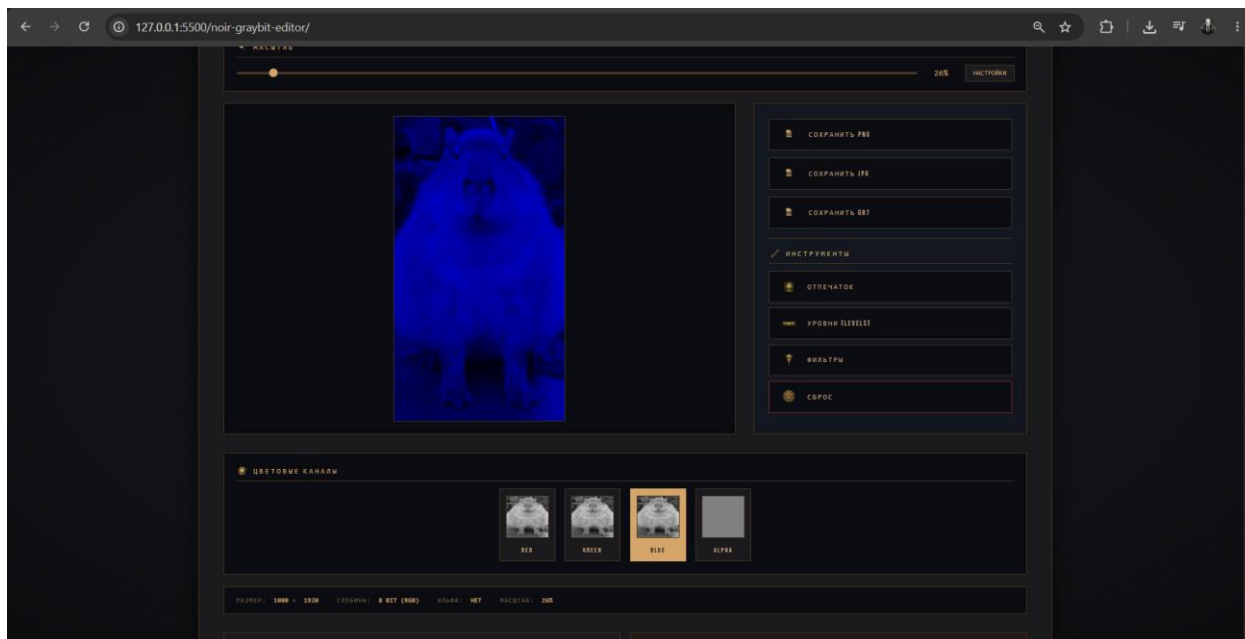


Рисунок 3 – канал BLUE

## 5.2 Инструмент «Пипетка»

Инструмент «Пипетка» представлен кнопкой в блоке «ИНСТРУМЕНТЫ». При активации кнопка подсвечивается, и курсор мыши при наведении на холст меняется на крестик. При клике по изображению на панели появляется информационный блок, отображающий координаты пикселя (X и Y), значения RGB (R, G, B) и значения CIELAB (L, a, b), а также цветовой образец выбранного пикселя. Информация автоматически обновляется при каждом клике.

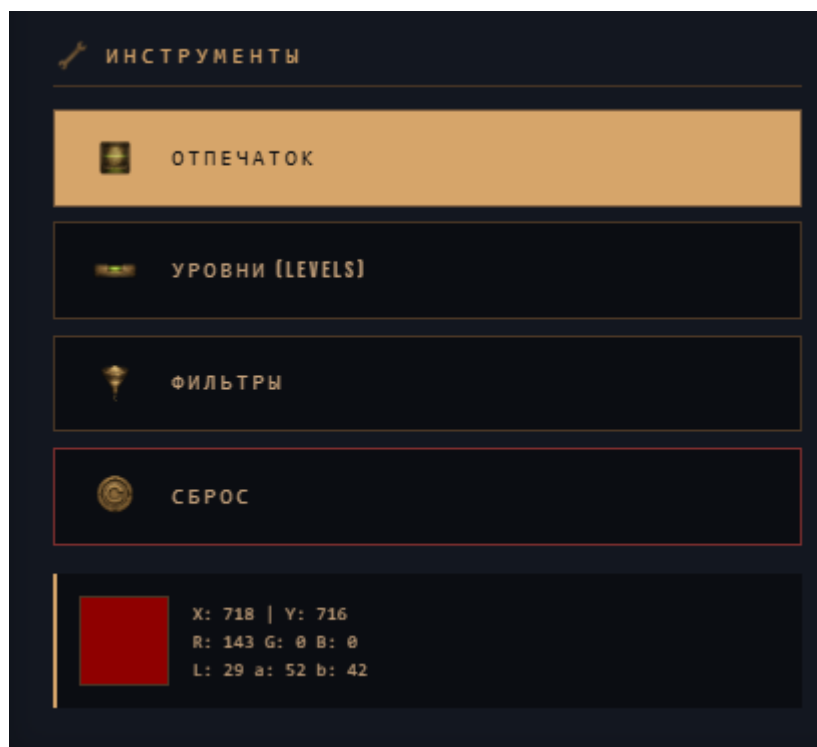


Рисунок 4 – использование Отпечатка ( Пипетки )

### **5.3 Визуальная обратная связь**

1. Активный канал - золотая рамка и фон
2. Неактивный канал - темная рамка, отсутствие подсветки
3. Активная пипетка - золотая кнопка, курсор-крестик
4. Информация о цвете - появляется панель с образцом цвета и числовыми значениями

## **6. Заключение**

В ходе выполнения лабораторной работы реализованы панель цветовых каналов (Red, Green, Blue, Alpha) с возможностью включения/выключения каждого канала и инструмент «Пипетка» для получения цвета пикселя с отображением координат X,Y, значений RGB и CIELAB. Также реализован конвертер RGB в CIELAB и нормализация миниатюр каналов. Приложение полностью соответствует требованиям.